BRNO UNIVERSITY OF TECHNOLOGY FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

Robotics and AI

MPC-MAP Assignment No. 4 - Report

Author: Martin Kříž

Date: 13.4.2026

Task 1

For the prediction step, I use the velocity motion model from Prob. Robotics, where Gaussian noise is added to the motion vector. The scale of the noise is controlled by parameters $\alpha_1 - \alpha_6$, which model the accuracy of the robot's motion. Less accurate robot requires larger α values. I implement a custom *my_randn()* function which returns a random sample from a normal distribution with a specified variance.

Task 2

For computing the simulated lidar measurement for each particle, I use the simulator's built-in *ray_cast()* function. To weigh the particles, I use rangefinder based on a normal distribution. For each ray, the likelihood is evaluated as $N(x; \mu, \sigma^2)$, where x is the simulated particle measurement, μ is the real lidar measurement, and σ is the per-ray sigma measured in Week 2. If a ray measurement is out of range, Prob. Robotics suggests the weight should be zero. However, if only a few rays are out of range, I choose to skip them rather than zeroing the entire particle weight, as the remaining valid rays still provide useful information.

Task 3

For resampling, I use the low variance sampler method, where a single random offset is drawn at the beginning.

Task 4

After the first attempt, I noticed particle degeneracy even with high noise in the prediction step (large α parameters). To address this, I added adaptive resampling, so the resampling is performed only every 4th iteration instead of every iteration.
For estimating the pose from $bel(x_t)$, which is a set of hypotheses about the robot's location, I use the weighted mean. For orientation, a circular mean is required to handle angle wrapping. I also tried using the particle with the highest weight as the pose estimate, but this proved unreliable, because randomly injected particles occasionally received high weights due to map symmetry, causing the estimate to jump to incorrect locations.
For the kidnapped robot problem, I use adaptive injection of random particles based on the Augmented MCL algorithm (8.3.3 Prob Robotics). Before weight normalization, slow and fast exponential filters track the measurement likelihood, and their ratio determines the number of randomly injected particles.
In the Appendix, Figure 1 shows the robot run without the Augmented MCL feature, and Figure 2 shows the robot run with this feature enabled.

The major issue is that if the robot does not acquire the correct pose at the beginning, it will likely never converge to the correct pose. I added a delayed resampling step (resampling starts only after the 20th iteration) which improved the results but did not fully resolve the issue.

Appendix

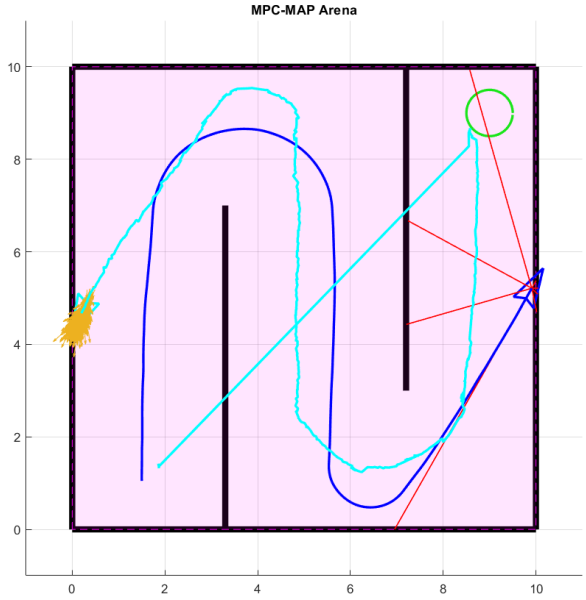

Fig. 1 – Example without AMLC

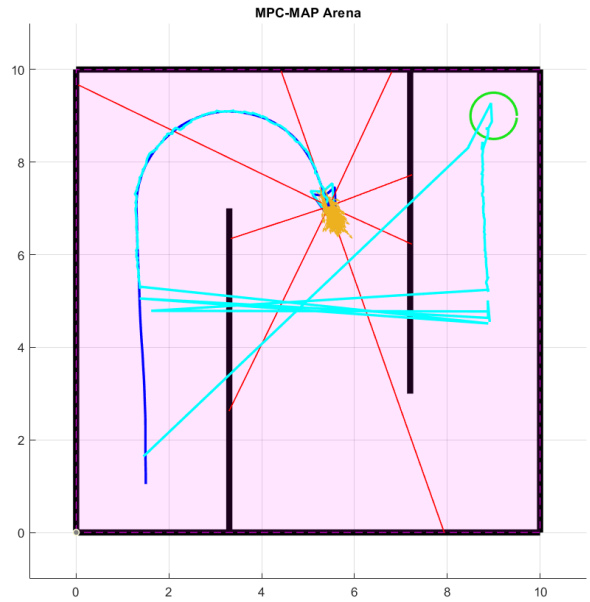

Fig. 2 – Example of the AMCL robot relocalization